

PROJETO 1

Sistema de Tickets Help Desk

Aplicacao web completa para gestao de incidentes IT

Autor	Helder Sindique
Data	Junho 2026
Stack	Python 3.12 / Flask / SQLite / Gunicorn
Deploy	helderlab.duckdns.org:5000
Repositorio	github.com/heldersindique-prog/ticket-system

1. Introducao

O Projeto 1 consiste no desenvolvimento de um sistema web completo de gestao de tickets de suporte IT, construido e deployado no servidor de laboratorio pessoal configurado no Projeto 0. O objetivo foi simular um ambiente real de Help Desk, com abertura de incidentes, classificacao automatica por prioridade, acompanhamento do estado e registo de resolucao.

O sistema foi desenvolvido de raiz em Python com Flask, sem recurso a frameworks de terceiros para o frontend, demonstrando conhecimento das tecnologias base. O deploy foi feito com Gunicorn como servidor WSGI de producao e configurado como servico systemd para arranque automatico com o servidor.

2. Objetivos

Objetivo	Resultado
Interface web funcional e acessivel remotamente	Acessivel em <code>helderlab.duckdns.org:5000</code>
Criacao e gestao de tickets de suporte	Formulario com categorias e classificacao automatica
Classificacao automatica de prioridade	P1/P2/P3 atribuido por categoria do incidente
Dashboard com metricas em tempo real	Contadores de total, abertos, fechados e P1 critico
Registo de resolucao e historico	Cada ticket tem data de abertura, fecho e nota de resolucao
Deploy com arranque automatico	Servico systemd ativo, reinicia com o servidor

3. Stack Tecnico

Componente	Tecnologia	Funcao
Backend	Python 3.12 / Flask	Rotas, logica de negocio, renderizacao de templates
Base de dados	SQLite	Persistencia de tickets e historico
Frontend	HTML/CSS/JS puro	Interface grafica sem dependencias externas
Servidor WSGI	Gunicorn	Servidor de producao com 2 workers
Servico	systemd	Arranque automatico e reinicio em caso de falha
Infraestrutura	Ubuntu 24.04	Servidor de laboratorio pessoal (Projeto 0)

4. Arquitetura do Sistema

O sistema segue uma arquitetura MVC simples. O ficheiro `models.py` centraliza toda a logica de base de dados. O ficheiro `app.py` define as rotas Flask. Os templates HTML constituem a camada de apresentacao.

Ficheiro	Responsabilidade
<code>app.py</code>	Rotas: <code>/</code> , <code>/novo</code> , <code>/ticket/</code> , <code>/fechar/</code> , <code>/api/stats</code>
<code>models.py</code>	SQLite: <code>init_db</code> , <code>criar_ticket</code> , <code>listar_tickets</code> , <code>fechar_ticket</code> , estatisticas
<code>templates/index.html</code>	Painel principal com dashboard e fila de tickets
<code>templates/novo.html</code>	Formulario de abertura de novo ticket
<code>templates/ticket.html</code>	Detalhe do ticket com formulario de fecho

5. Logica de Classificacao de Prioridade

A prioridade e atribuida automaticamente com base na categoria selecionada, sem intervencao manual. Esta logica replica o comportamento de sistemas ITSM reais onde certas categorias implicam SLAs diferentes.

Categoria	Prioridade	Justificacao
Rede	P1 - Critico	Impacto em multiplos utilizadores simultaneamente
Acesso	P1 - Critico	Utilizador bloqueado, impacto imediato na producao
Hardware	P2 - Alto	Equipamento com falha, requer intervencao tecnica
Software	P2 - Alto	Aplicacao com erro, impacto na produtividade
Impressora	P3 - Normal	Servico nao critico, pode aguardar agendamento

Figura 1 - Servidor Flask/Gunicorn iniciado e a correr na porta 5000

Figura 2 - Painel principal com dashboard de contadores em tempo real

Figura 3 - Fila de tickets com 3 incidentes e prioridades atribuídas automaticamente

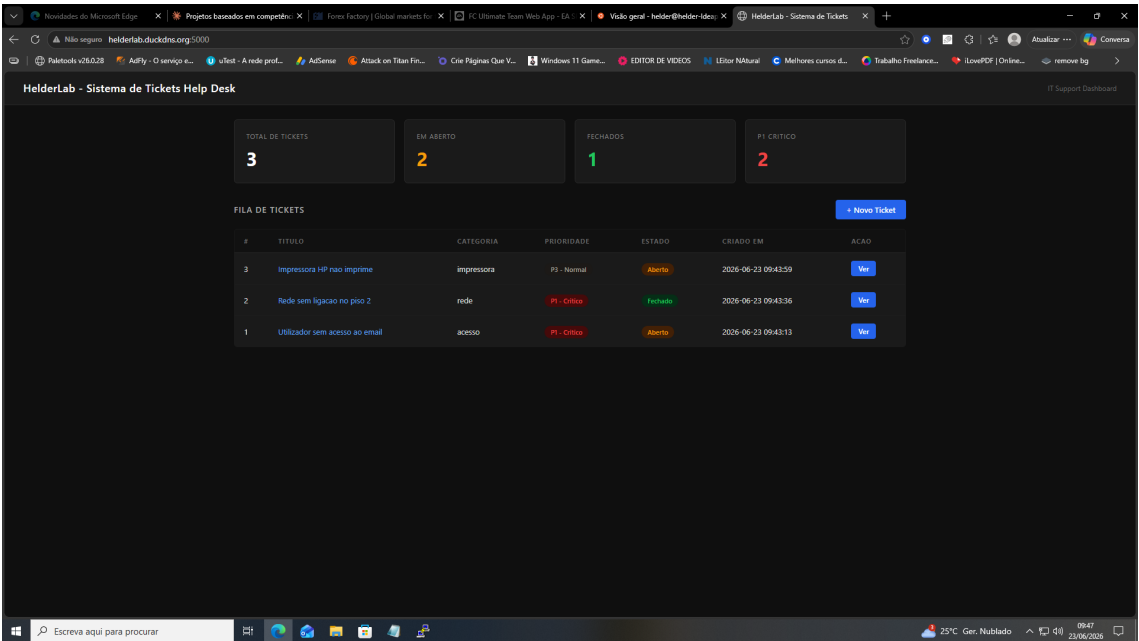


Figura 4 - Ticket #2 fechado: painel atualiza contadores em tempo real

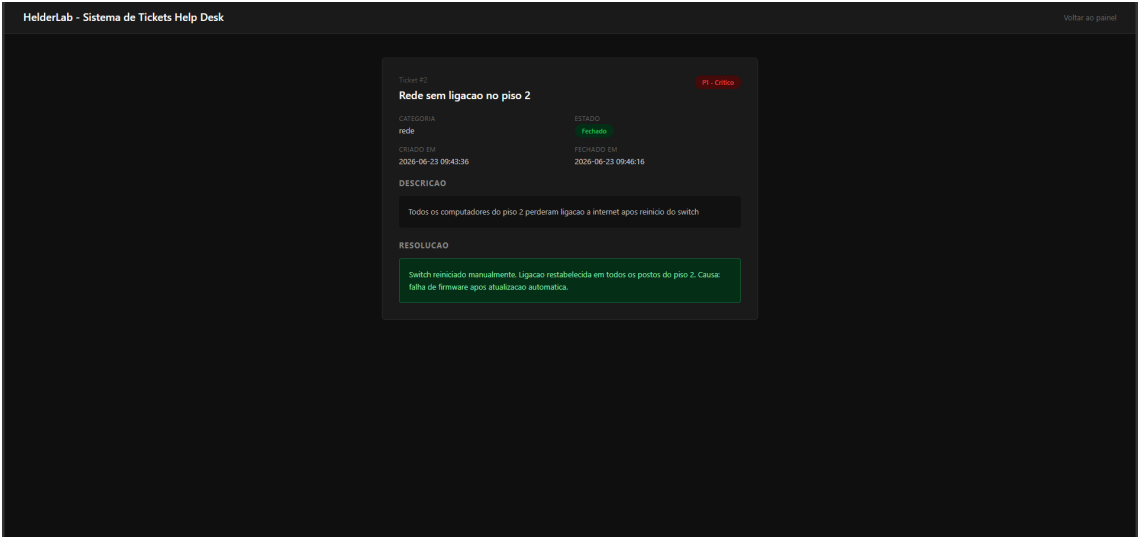


Figura 5 - Detalhe do Ticket #2 com descricao, resolucao e datas documentadas

7. API REST de Estatísticas

O sistema expoe um endpoint REST em /api/stats que retorna estatísticas em formato JSON, podendo ser consumido por ferramentas de monitorização externas ou integrado em dashboards futuros.

```
{"total": 3, "abertos": 2, "fechados": 1, "por_categoria": {"acesso": 1, "impressora": 1, "rede": 1}, "por_prioridade": {"P1 - Critico": 2, "P3 - Normal": 1}}
```

8. Deploy e Operação

O sistema corre em produção com Gunicorn configurado com 2 workers. O serviço systemd garante arranque automático e reinício em caso de falha.

Parametro	Valor
Servidor WSGI	Gunicorn 2 workers
Porta	5000/TCP
Bind	0.0.0.0 (todas as interfaces)
Serviço systemd	ticket-system.service (enabled)
URL pública	http://helderlab.duckdns.org:5000
Reinício automático	Restart=always
Base de dados	tickets.db (SQLite, ficheiro local)

9. Conclusão

O Projeto 1 demonstra competências práticas diretamente relevantes para funções de IT Technician e Help Desk: compreensão do fluxo de gestão de incidentes, classificação por prioridade, registo de resoluções e operação de sistemas web em ambiente Linux.

Do ponto de vista técnico, o projeto cobre desenvolvimento backend em Python, persistência com SQLite, interface web sem dependências externas, deploy com Gunicorn e gestão de serviços com systemd. O código está publicamente disponível no GitHub e o sistema está acessível em produção.

Componente	Estado
Aplicação Flask	Funcional em produção
Base de dados SQLite	Persistente com histórico completo
Classificação automática de prioridade	P1/P2/P3 por categoria
Dashboard com contadores	Funcional em tempo real
Fecho de tickets com resolução	Funcional
API REST /api/stats	Funcional, retorna JSON
Deploy Gunicorn	2 workers, porta 5000
Serviço systemd	Enabled, arranque automático
Repositório GitHub	github.com/heldersindique-prog/ticket-system
URL pública	http://helderlab.duckdns.org:5000